

Karate Courses Frontend — Complete Project Guide

Last reviewed against the repository: 23 June 2026

Application: Sensei Yari Karate Academy course storefront and student portal

Scope: this repository contains the **Next.js frontend**. Authentication, database, email/OTP, course ownership, and payment operations require a separate backend.

Table of contents

1. [Project overview](#)
2. [Quick start](#)
3. [Demo accounts and roles](#)
4. [Technology stack](#)
5. [Frontend architecture](#)
6. [Directory structure](#)
7. [Routes and access rules](#)
8. [Core user flows](#)
9. [State management and persistence](#)
10. [Backend integration and API contract](#)
11. [Data models and validation](#)
12. [Authentication, authorization, and security](#)
13. [Internationalization](#)
14. [UI, styling, assets, and responsive behavior](#)
15. [Rendering and data-fetching strategy](#)
16. [Environment variables](#)
17. [Scripts and developer workflow](#)
18. [Production build and deployment](#)
19. [Testing and verification](#)
20. [Troubleshooting](#)
21. [Known constraints and maintenance notes](#)
22. [Contribution checklist](#)

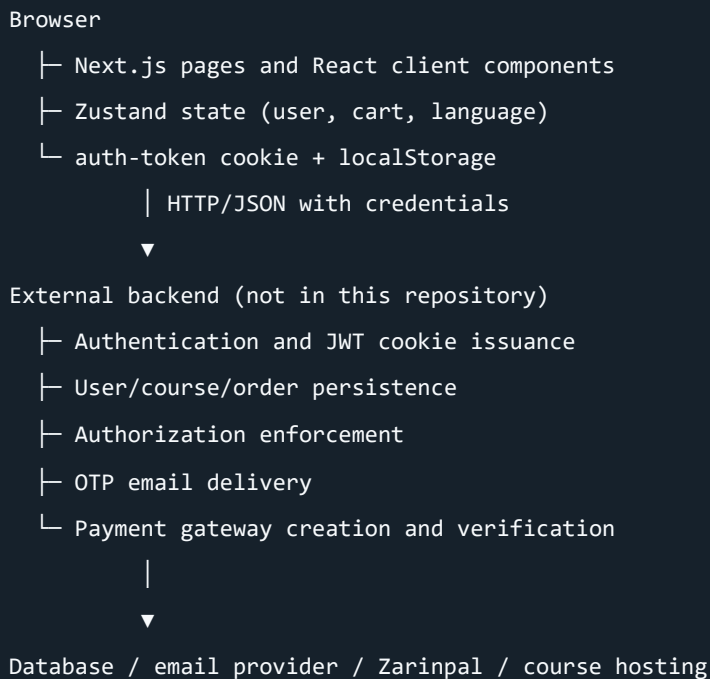
1. Project overview

This application is the public website and account portal for an online karate course platform. It provides:

- A bilingual Persian/English landing page.
- A course catalog and individual course pages.
- Browser-persisted shopping cart functionality.
- Registration, login, logout, password recovery with OTP, and password change.
- A protected user profile with editable account information.
- Access to purchased course links.
- Checkout and Zarinpal-style payment redirection/verification.
- An administrator panel for creating/deleting courses and looking up a user's purchased courses.
- Newsletter registration.

The frontend does **not** contain the backend server, database schema, mail service, or payment gateway implementation. It communicates with the backend configured by `NEXT_PUBLIC_BACKEND_URL`.

System boundary



2. Quick start

Prerequisites

- Node.js 20 LTS is recommended. Next.js 14 requires Node.js 18.17 or newer.
- npm (the repository includes `package-lock.json`; use npm for reproducible installs).
- Git.
- The separate backend running and reachable. The current local convention is `http://localhost:3001`.

Install and run locally

```
git clone <repository-url>
cd karate-courses
npm ci
```

Create `.env.local`:

```
NEXT_PUBLIC_BACKEND_URL=http://localhost:3001
```

Start the backend first, then start this frontend:

```
npm run dev
```

Open `http://localhost:3000`.

Use `npm ci` for clean/CI installations because it installs the exact versions in `package-lock.json`. Use `npm install` only when intentionally changing dependencies.

Basic smoke test

1. Open the landing page.
2. Visit `/courses` and confirm that backend courses appear.
3. Register a normal user, then log in.
4. Add a course to the cart and open checkout.
5. Open `/profile` and confirm profile data and purchased courses.
6. Log in with the demo administrator account and open `/profile/admin`.

3. Demo accounts and roles

Administrator (development/demo)

Field	Value
Email	<code>ali@gmail.com</code>
Password	<code>Ali13831383</code>

Use this account to exercise administrator-only UI, including course creation, course deletion, and user-course lookup.

Security warning: these are shared demo credentials. They must not protect a production environment. Replace the password, store credentials outside the repository, and have the backend seed/administer production accounts securely.

Normal users

Normal users must register themselves at `/auth/register`; there is no shared normal-user account. Registration requires first name, last name, email, and password. After successful registration, the UI redirects to `/auth/login`.

Role behavior

- Guests can browse the landing page and course catalog and can maintain a local cart.
- Authenticated users can access profile, checkout, payment verification, and purchased courses.
- Administrators receive an `isAdmin: true` claim in the JWT and can access `/profile/admin`.
- The course detail page shows delete/admin actions to administrators and add-to-cart behavior to non-administrators.

4. Technology stack

Area	Technology	Purpose
Framework	Next.js 14.2.4, App Router	Routing, server/client rendering, middleware, metadata
Language	TypeScript 5, strict mode	Static typing
UI runtime	React 18	Component model and hooks
Component library	Chakra UI 2	Layout, forms, dialogs, tables, responsive props
Utility CSS	Tailwind CSS 3 + PostCSS	Utility classes and global styling
Animation	Framer Motion	Landing-page motion
State	Zustand 4	Auth mirror, cart, and language stores
HTTP	Axios	Credentialed backend requests
Forms	React Hook Form	Form state and submission
Validation	Zod + Hook Form resolver	Client-side schemas and messages
Notifications	React Toastify	Success/error feedback
Authentication helpers	<code>jwt-decode</code> , <code>cookie</code>	JWT claim reading and cookie-related types
Images	Next Image, Chakra Image, Sharp	Image display and production optimization support
Icons	React Icons + Chakra icons	UI iconography

The project deliberately combines Chakra UI and Tailwind. Chakra is the main component/layout system; Tailwind utilities are used for compact spacing, sizing, typography, and flex/grid adjustments.

5. Frontend architecture

Architectural layers

src/app	Route tree, root layout, server pages, route composition
↓	
src/components	Feature UI and client-side interaction
src/layouts	Reusable structural wrappers and provider composition
↓	
src/stores	Browser state and persistence
src/services	Axios instance, typed API wrapper, server action
↓	
src/schemas	Zod validation and TypeScript data types
src/utils	JWT, URL, image, password, and metadata helpers
src/lang	Persian and English dictionaries

Server and client component split

- Components without `"use client"` run as React Server Components by default.
- Course catalog and individual course data are fetched from server-rendered route components.
- Interactive forms, cart, navbar, language switching, payment verification, and profile widgets are client components.
- The root layout is a server component and wraps the application with the client-side Chakra provider.
- `registerNewsletter` is a server action (`"use server"`) used by `useFormState` .

Shared providers and shell

`src/app/layout.tsx` establishes:

- Local IRANSans and Lalezar fonts.
- Global CSS and Tailwind layers.
- Chakra's `ChakraProvider` through `src/layouts/Providers.tsx` .
- Navbar and footer around every route.
- A global RTL Toastify notification container.
- Persian metadata and `<html lang="fa">` as the initial document language.

6. Directory structure

```
karate-courses/
├─ public/                Static WebP images and branding
├─ src/
│  ├─ app/                App Router routes, root layout, CSS, local fonts
│  │  ├─ auth/            Login, registration, forgot-password flow
│  │  ├─ courses/         Catalog and dynamic course detail route
│  │  ├─ payment/         Checkout and gateway callback verification
│  │  └─ profile/         User profile and nested admin panel
│  ├─ components/         Navbar, footer, landing, cart, course and user UI
│  │  └─ Form/            Feature forms and reusable form primitives
│  ├─ lang/               `fa` and `en` translation dictionaries
│  ├─ layouts/            Panels, form wrappers, background, providers
│  ├─ schemas/            Course/payment/auth Zod schemas and types
│  ├─ services/           Axios client and newsletter server action
│  ├─ stores/             Auth, cart, and language Zustand stores
│  ├─ utils/              Focused transformation/validation helpers
│  └─ middleware.ts        Route protection and role redirects
├─ next.config.mjs        Next image configuration
├─ tailwind.config.ts     Tailwind scanning and custom colors
├─ theme.ts               Empty Chakra theme extension placeholder
├─ tsconfig.json          Strict TS and `@/*` → `src/*` alias
├─ package.json           Scripts and dependencies
└─ .env / .env.local      Runtime configuration (see security notes)
```

Important component responsibilities

- `LandingPage` : large marketing homepage and role-aware calls to action.
- `Navbar` / `BurgerMenu` : responsive navigation, auth actions, cart, language selector.
- `CourseCard` : catalog presentation and course-detail link.
- `AddToCartBtn` : checks purchased courses, then offers access or cart addition.
- `Cart` : persistent drawer and checkout navigation.
- `Checkout` : builds the payment request and redirects to the returned gateway URL.
- `UserCourses` : loads the current user's courses or, for admins, courses by email.
- `AdminForm` : creates courses; `DeleteBtn` removes a course after confirmation.
- `ProfileForm` / `ChangePasswordForm` : account maintenance.

7. Routes and access rules

Route	Rendering	Access	Responsibility
/	Static	Public	Landing/marketing page
/courses	Forced static	Public	Fetch and list all courses; newsletter form
/courses/[courseId]	Forced dynamic	Public	Course detail; role-aware course action
/auth/login	Static	Guests only	Login and auth-cookie creation through backend
/auth/register	Static	Guests only	New normal-user registration
/auth/forget-password	Static client flow	Guests only	Email → OTP validation → new password
/profile	Dynamic/no-store	Authenticated	Profile edit, password change, purchased courses
/profile/admin	Dynamic/no-store	Admin only	Course creation and user-course lookup
/payment/checkout	Dynamic/no-store	Authenticated	Cart summary and payment initialization
/payment/verify	Static shell + client verification	Authenticated	Validate gateway callback and display order

`src/middleware.ts` applies to `/profile/:path*`, `/auth/:path*`, and `/payment/:path*`:

- Missing `auth-token` on profile/payment routes redirects to `/auth/login`.
- An authenticated user visiting `/auth/*` is redirected to `/profile`.
- A non-admin JWT visiting `/profile/admin` is redirected to `/profile`.

8. Core user flows

Registration and login

1. A normal user submits `POST /register`.
2. On success, the frontend redirects to `/auth/login`.
3. Login submits `POST /login` with credentials enabled.
4. The backend must set the `auth-token` cookie and return a public `user` object.
5. The user object is mirrored into browser local storage by the auth store.
6. Middleware uses the cookie; interactive UI uses the local user mirror.

Password recovery

1. `PUT /forget-password` sends the email.
2. The current frontend expects an `OTP` value in the response and shows it in a toast.
3. `POST /validate-otp` submits `{ OTP, email }`.
4. `PUT /reset-password` submits matching new passwords.
5. The app logs out and redirects to login.

For production, OTP should be delivered out-of-band by email/SMS and should not be returned to or displayed by the client.

Cart and checkout

1. Course detail calls `GET /user/fetch-course` to detect existing ownership.
2. A non-owned course can be added to the Zustand cart.
3. Cart contents are saved in local storage under `cart`.
4. Checkout submits `POST /payment/checkout` with string course IDs.
5. The backend returns a `paymentUrl`; the browser performs a full redirect.
6. The gateway returns to `/payment/verify?Authority=...&Status=...`.
7. For `Status=OK`, the frontend calls `POST /payment/verify` with `authority`.
8. The verified order and external course links are displayed.

The backend must calculate prices and course eligibility itself; never trust cart prices or IDs solely because they came from the frontend.

Admin course management

1. Log in with an administrator account.
2. Open `/profile/admin` from the profile card.
3. Submit title, description, numeric price, base64 image, and course link to `POST /create-course`.
4. Open a course detail page and use the delete action, which calls `DELETE /delete-course/:courseId`.
5. Search by email to call `GET /admin/fetch-course/:email` and inspect that user's purchased courses.

9. State management and persistence

Auth store (`src/stores/authStore.ts`)

- Key: `user` in local storage.
- State: `{ email, firstName, lastName } | null`.
- `login` persists the public user object.
- `logout` clears local storage, calls `POST /logout`, shows a toast, then reloads `/`.
- The cookie is not set by this store; it must be set/cleared by the backend.

Cart store (`src/stores/cartStore.ts`)

- Key: `cart` in local storage.
- Stores complete course objects for display convenience.
- Prevents duplicate IDs, supports remove/clear, and exposes a hydration flag.
- Persistence is browser-specific and not synchronized to the backend.

Language store (`src/stores/languageStore.ts`)

- Key: `language` in local storage.
- Values: `fa` or `en`.
- Exposes the active dictionary as `t`.

Hydration pattern

Persisted browser state is unavailable during server rendering. Components such as `Navbar`, `Cart`, profile forms, checkout, and user courses delay browser-dependent output until mounted, often showing placeholders/spinners. Follow this pattern when adding browser-persisted state to avoid hydration mismatches.

10. Backend integration and API contract

All calls use the Axios instance in `src/services/httpService.ts`:

- Base URL: `NEXT_PUBLIC_BACKEND_URL`.
- JSON content type.
- `withCredentials: true`, allowing cookies on browser requests.
- No configured timeout or interceptor.

`ApiClient<T>` adds generic `get`, `post`, `put`, and `delete` methods around one endpoint. The generic currently describes request/domain data but does not strongly type every response envelope.

Inferred endpoints

The following contract is inferred from frontend usage; the backend remains the source of truth.

Method	Endpoint	Request	Frontend expects
POST	<code>/register</code>	<code>{ firstName, lastName, email, password }</code>	<code>{ message }</code>
POST	<code>/login</code>	<code>{ email, password }</code>	<code>{ message, user }</code> + <code>auth-token</code> cookie
POST	<code>/logout</code>	—	<code>{ message }</code> and cleared cookie

Method	Endpoint	Request	Frontend expects
PUT	/forget-password	{ email }	{ OTP, message? }
POST	/validate-otp	{ OTP: number, email }	{ message }
PUT	/reset-password	{ newPassword, repeatPassword }	{ message }
PUT	/profile	{ firstName, lastName, email }	{ message }; refreshed claims/cookie may be needed
GET	/fetch-course	—	Course[]
GET	/fetch-course/:id	—	Course
POST	/create-course	{ title, description, price, img, link }	{ message }
DELETE	/delete-course/:id	—	{ message }
GET	/user/fetch-course	Cookie	Purchased Course[]
GET	/admin/fetch-course/:email	Admin cookie	User's purchased course array
POST	/payment/checkout	{ courseIds: string[] }	{ message, paymentUrl }
POST	/payment/verify	{ authority }	{ courses, transaction, message? }
POST	/register-newsletter	{ email }	{ message }

Backend requirements

- Parse JSON request bodies.
- Allow the frontend origin with credentialed CORS. With credentials, `Access-Control-Allow-Origin` must be the explicit frontend origin, not `*`.
- Set a cookie named exactly `auth-token` with an appropriate domain/path/SameSite/Secure policy.
- Return a JWT containing `email`, `firstName`, `lastName`, and boolean `isAdmin` claims.
- Enforce authentication and administrator authorization independently of the UI and middleware.
- Validate all inputs and return human-readable `{ message }` error bodies where possible.
- Recompute order totals and verify payment authority server-side.

The `Access-Control-Allow-*` values currently attached to Axios request headers do not configure browser CORS; CORS must be configured on the backend response.

11. Data models and validation

Public course

```
{
  id: string;
  title: string;      // 5-50 characters
  description: string; // 20-150 characters in Course schema
  price: number;      // non-negative
  img: string;
}
```

Course creation

Adds `link: string`, permits descriptions up to 1000 characters, and expects `img` as a string. Images may be:

- A `data:image/...` URI.
- An HTTP(S) URL.
- A root-relative public path.
- Raw base64; the helper infers PNG, WebP, GIF, or defaults to JPEG.

Course links missing a scheme are normalized to `https://`.

User/JWT claims

```
{
  email: string;
  firstName: string;
  lastName: string;
  isAdmin: boolean;
}
```

Password rule

The shared regex is `^(?=.*[a-z])(?=.*\d){8,}$`:

- At least 8 characters.
- At least one lowercase English letter.
- At least one digit.

Despite some UI wording saying “one letter,” uppercase-only or Persian-only passwords do not satisfy the current regex.

Payment request and response

Checkout sends at least one string course ID. Verification expects:

```
{
  courses: CreateCourseType[];
  transaction: {
    transactionId: string;
    isPaid: boolean;
    totalPrice: number;
  };
}
```

12. Authentication, authorization, and security

Current model

- `auth-token` cookie is the route-access signal.
- Middleware decodes the JWT to read `isAdmin` but does not cryptographically verify it.
- Local-storage `user` controls interactive display only.
- All Axios requests include credentials.

Mandatory production rules

- The backend must verify JWT signature, expiry, issuer/audience as applicable, and role on every protected API.
- Use `HttpOnly`, `Secure` in HTTPS, and a deliberate `SameSite` policy for the auth cookie.
- Add CSRF protection if cross-site credentialed requests are possible.
- Never authorize based on the frontend's `isAdmin` display state.
- Sanitize and validate course URLs and base64 image size/type server-side.
- Do not expose OTPs in API responses in production.
- Apply login/OTP rate limits and avoid account-enumeration messages.
- Rotate the documented demo password before a public deployment.
- Store payment merchant credentials only in the backend/hosting secret store. Any variable prefixed `NEXT_PUBLIC_` is bundled for browsers.

Repository secret hygiene

The root `.env` is currently tracked by Git, while `.gitignore` only ignores `.env*.local`. Treat any value ever committed there as exposed: rotate real secrets, remove them from the tracked file/history as appropriate, and use `.env.local` locally plus hosting-platform secrets in production. Commit only a safe `.env.example` if a template is needed.

13. Internationalization

Translation dictionaries are in `src/lang/fa.ts` and `src/lang/en.ts`. `useLanguageStore()` provides `t` and persists the chosen language.

Current coverage is partial:

- The landing page, navbar, profile fields, and several common actions use dictionaries.
- A number of course, checkout, admin, password-recovery, footer, toast, and validation strings remain hard-coded in Persian.
- Root metadata, document `lang`, and Toastify `rtl` remain Persian even after switching to English.

When adding a string, add the same key to both dictionaries and use `t` rather than embedding text. A complete i18n implementation should also update document `lang`, `dir`, metadata, and toast direction based on the active locale, ideally with locale-aware routes or server-readable locale state.

14. UI, styling, assets, and responsive behavior

- Chakra responsive props use breakpoints such as `base`, `sm`, `md`, and `lg`.
- Tailwind scans `src/pages`, `src/components`, `src/layouts`, and `src/app`.
- Custom Tailwind colors are `primary: #ff0000` and `heading: #0d161b`.
- `theme.ts` is currently an empty Chakra extension point.
- Local fonts reside in `src/app/fonts`; IRANSans is the primary sans font and Lalezar is used for display headings.
- Static images live under `public/`.
- Next remote image configuration permits `https://placeholder.co/**`; course images also use `unoptimized`, allowing URL/data sources without Next optimization.
- Navigation adapts between desktop and burger-menu layouts; the cart drawer opens at the bottom on mobile and right side on larger screens.
- The design frequently uses dark translucent panels, backdrop blur, red action colors, and teal profile/course accents.

15. Rendering and data-fetching strategy

- `/courses` exports `dynamic = "force-static"`. It calls `/fetch-course` while building/prerendering.

- `/courses/[courseId]` is forced dynamic because it reads cookies and course-specific backend data.
- `/profile`, `/profile/admin`, and `checkout` are dynamic/no-store.
- Payment verification reads query parameters on the client inside `Suspense`.
- Browser-only data is explicitly hydrated after mount.

Course catalog build implication

The production build can complete while the backend is unavailable because the catalog catches fetch errors and renders an empty array. That means:

- CI may show `ECONNREFUSED` but still exit successfully.
- A deployment built without backend access can ship an empty static catalog.
- Course changes will not automatically appear until the page is rebuilt/revalidated under the current forced-static strategy.

If courses must always be current, consider dynamic rendering, timed revalidation, or explicit on-demand revalidation after admin changes.

16. Environment variables

Frontend variable

Variable	Required	Visibility	Description
<code>NEXT_PUBLIC_BACKEND_URL</code>	Yes	Browser and server bundle	Absolute backend base URL, e.g. <code>http://localhost:3001</code>

Backend-only variable

Variable	Location	Description
<code>ZARINPAL_MERCHANT_ID</code>	Backend secret store	Merchant identity for payment requests; this frontend source does not read it

Suggested local `.env.local`:

```
NEXT_PUBLIC_BACKEND_URL=http://localhost:3001
```

Do not add a payment secret to a `NEXT_PUBLIC_*` variable. Restart `next dev` after changing environment variables.

17. Scripts and developer workflow

Command	Purpose
<code>npm ci</code>	Exact clean dependency installation
<code>npm run dev</code>	Start Next development server
<code>npm run build</code>	Compile, type-check, lint, and generate production output
<code>npm run start</code>	Serve the existing production build
<code>npm run lint</code>	Run the Next.js ESLint integration

Recommended change cycle:

```
npm ci
npm run dev
# implement and test the change
npm run lint
npm run build
```

The `@/*` path alias resolves to `src/*`. Keep feature code within the existing layer boundaries and prefer the shared form, panel, API, schema, and URL/image helpers.

18. Production build and deployment

Generic Node deployment

```
npm ci
npm run build
npm run start
```

The default production port is 3000. Set `PORT` at process level if the hosting environment requires another port.

Deployment checklist

- Configure `NEXT_PUBLIC_BACKEND_URL` before building; public variables are compiled into client bundles.
- Ensure the backend is reachable during the build if the static course catalog must contain data.
- Configure the backend's allowed frontend origin and credentialed CORS.
- Configure the auth cookie for the production frontend/backend domains.

- Configure the payment callback URL to end at `/payment/verify` and preserve `Authority` and `Status` query parameters.
- Serve over HTTPS.
- Rotate/remove demo and committed credentials.
- Run `npm run build` in CI.
- Confirm external course links and image payload sizes.

Verified repository status

On 23 June 2026, `npm run build` completed successfully with Next.js 14.2.4. Because the local backend on port 3001 was not running, the build logged `ECONNREFUSED` for `/fetch-course`, but the catalog's error handling allowed the build to finish.

19. Testing and verification

There is currently no automated unit, integration, or end-to-end test suite in the repository. Until one is added, verify at least:

- Public rendering at mobile and desktop widths.
- Persian/English switch and persistence after reload.
- Registration validation and successful redirect.
- Login cookie behavior, logout, and auth-route redirect.
- Guest redirect from profile/payment routes.
- Non-admin redirect away from `/profile/admin`.
- Profile edit and password change.
- Course list/detail, image variants, duplicate cart prevention, remove/clear.
- Empty and populated checkout.
- Payment cancellation, missing query parameters, backend verification failure, and success.
- Admin create/delete and user-email lookup.
- Backend downtime behavior and readable error messages.

Recommended future tooling: Vitest + React Testing Library for units/components, MSW for API mocks, and Playwright for role/payment navigation flows.

20. Troubleshooting

Courses are empty or build logs `ECONNREFUSED localhost:3001`

- Start the backend on port 3001, or update `NEXT_PUBLIC_BACKEND_URL`.
- Restart the frontend after environment changes.

- For production builds, make the backend reachable during prerendering or change the catalog rendering strategy.

Login succeeds but protected routes redirect to login

- Inspect the response and confirm a cookie named `auth-token` was set.
- Check cookie domain, path, expiry, SameSite, and Secure flags.
- Confirm browser requests include credentials and backend CORS allows the exact frontend origin.

CORS errors

- Fix CORS on the backend; client `Access-Control-Allow-*` request headers do not grant permission.
- For credentialed requests, do not return wildcard `Access-Control-Allow-Origin: *`.

Admin route redirects to profile

- Decode the issued JWT and confirm it includes boolean `isAdmin: true`.
- Confirm the cookie contains the current token and is visible to Next middleware.
- Still enforce admin access on the backend, regardless of middleware behavior.

Cart/user/language content appears late

This is expected during local-storage hydration. If it never appears, inspect local storage values for malformed JSON and check browser console errors.

`npm run lint` reports an `.next/cache/eslint` permission error

- Ensure another build/lint process is not using `.next`.
- Close processes holding the file and run lint again.
- Avoid launching lint and build concurrently because both write to `.next`.

Payment returns an error

- Confirm the gateway returns both `Authority` and `Status`.
- Only `Status=OK` triggers backend verification.
- Confirm `/payment/verify` accepts `{ authority }`, validates it with the gateway, and returns the expected transaction/course shape.

21. Known constraints and maintenance notes

- Frontend and backend are tightly coupled through inferred response shapes without a generated/shared API contract.
- JWT middleware decodes but does not verify the token; it is navigation convenience, not a security boundary.

- The auth UI mirror can become stale if the cookie changes elsewhere.
- Profile updates may require the backend to refresh JWT claims, otherwise server-rendered profile claims can lag behind local state.
- Course create and public course schemas allow different maximum description lengths (1000 vs 150).
- The forced-static catalog can become stale and can silently build empty.
- Cart toasts currently use Persian dictionary entries directly in the store for some actions.
- Translation coverage and document direction switching are incomplete.
- OTP is currently expected in a frontend-visible response.
- No API timeout, retry policy, centralized error normalization, error boundary, or automated tests are configured.
- `ZARINPAL_MERCHANT_ID` is not consumed by this frontend and belongs in backend configuration.
- The checked-in `.env` policy needs hardening.

These notes describe the current implementation; they are not requirements to preserve problematic behavior.

22. Contribution checklist

Before opening a change:

- Keep secrets and real credentials out of commits.
- Add/update Zod schemas with form or API shape changes.
- Add both Persian and English strings.
- Check server/client component boundaries before using browser APIs.
- Preserve hydration guards for local-storage state.
- Enforce permissions on the backend, even if UI/middleware also checks them.
- Verify mobile and desktop layouts.
- Verify guest, user, and admin behavior.
- Run `npm run lint` and `npm run build` sequentially.
- Update this guide and its Persian counterpart when routes, environment variables, API endpoints, or workflows change.